CAPACITAÇÃO EM IOT E ANDROID EMBARCADO

Android Embarcado e Framework

Tópico 4: Framework e Comunicação IPC (AIDL/Binder)
Prof. Isaías Felzmann - IC/Unicamp

Aluno(a)s: Pedro Araújo, Henrique Santos, Pedro Franco e Pedro Ramos

Prática 4 - Comunicação com HAL via AIDL/Binder

Objetivo: *O objetivo dessa atividade é criar uma interface AIDL que exponha dados provenientes da HAL do Acelerômetro.*

Etapa 0 – Preparação do ambiente

1. Acesse a plaquinha via adb:

```shell
adb pair <IP_PLACA>
adb root
adb remount
```

Estrutura Esperada do Projeto

```shell
hardware/
└── accel_daemon_<nome_grupo>/
    ├── 📄 Android.bp
    ├── 📄 accel_<nome_do_grupo>.rc
    ├── 📂aidl/
    │   └── 📂com/
    │       └── 📂android/
    │           └── 📂accel_<nome_grupo>/
    │               └── 📄 IAccel.aidl
    └── 📂src/
        └── 📄 accel_daemon_<nome_grupo>.cpp
```

Etapa 1 – Definição da interface AIDL

1. Criar estrutura do projeto:

```shell
cd /data/aosp/android_course_environment/hardware/
mkdir acell_daemon_<nome_grupo>
cd acell_daemon_<nome_grupo>
mkdir -p aidl/com/android/accel_<nome_grupo>
```

2. Criar arquivo AIDL:

```shell
touch aidl/com/android/accel_<nome_grupo>/IAccel.aidl
```

3. Definir a interface dentro de IAccel.aidl:

```C/C++
package com.android.accel_<nome_grupo>;
interface IAccel {
    float getAccelX();
    float getAccelY();
    float getAccelZ();
}
```

Etapa 2 — Implementação do Serviço Nativo (Daemon)

1. Criar diretório de implementação:

```shell
mkdir src
```

2. Criar arquivo do serviço:

```shell
touch src/accel_daemon_<nome_grupo>.cpp
```

3. Implementação do serviço:

```C/C++
#include <android/binder_manager.h>
#include <android/binder_process.h>
```

```
#include <android-base/logging.h>
#include <aidl/com/android/accel_froyo/BnAccel.h>
#include "AcelerometroHAL.h"

using aidl::com::android::accel_froyo::BnAccel;
using ndk::ScopedAStatus;
class AccellService : public BnAccel {
private:
    AcelerometroHAL* accel;
public:
    AccellService() {
        accel = new AcelerometroHAL();
        if (accel->init()) {
            LOG(INFO) << "HAL do Acelerômetro' inicializada com sucesso.";
        } else {
            LOG(ERROR) << "Falha ao inicializar a HAL.";
        }
    }
    ~AccellService() {
        delete accel;
    }
    ScopedAStatus getAccelX(float* _aidl_return) override {
        if (accel) {
            IMUData data = accel->readAccel();
            LOG(INFO) << "Accel X: " << data.x;
            *_aidl_return = data.x;
        } else {
            *_aidl_return = 0.0f;
            LOG(ERROR) << "Falha ao ler aceleração X. HAL não inicializada.";
        }
        return ScopedAStatus::ok();
    }
    ScopedAStatus getAccelY(float* _aidl_return) override {
        if (accel) {
            IMUData data = accel->readAccel();
            LOG(INFO) << "Accel Y: " << data.y;
            *_aidl_return = data.y;
        } else {
            *_aidl_return = 0.0f;
```

```
            LOG(ERROR) << "Falha ao ler aceleração Y. HAL não inicializada.";
        }
        return ScopedAStatus::ok();
    }
    ScopedAStatus getAccelZ(float* _aidl_return) override {
        if (accel) {
            IMUData data = accel->readAccel();
            LOG(INFO) << "Accel Z: " << data.z;
            *_aidl_return = data.z;
        } else {
            *_aidl_return = 0.0f;
            LOG(ERROR) << "Falha ao ler aceleração Z. HAL não inicializada.";
        }
        return ScopedAStatus::ok();
    }
};
int main() {
  android::base::InitLogging(nullptr,
android::base::LogdLogger(android::base::SYSTEM));
    LOG(INFO) << "Iniciando o daemon do acelerômetro...";

    ABinderProcess_setThreadPoolMaxThreadCount(1);
    ABinderProcess_startThreadPool();

    std::shared_ptr<AccellService> service =
        ndk::SharedRefBase::make<AccellService>();
    binder_status_t status =
        AServiceManager_addService(service->asBinder().get(), "acell_<nome_do_grupo>");
    if (status != STATUS_OK) {
        LOG(ERROR) << "Falha ao registrar serviço!";
        return -1;
    }
    LOG(INFO) << "Serviço registrado como 'acell_native'";
    ABinderProcess_joinThreadPool();
    return 0;
}
```

Esse serviço é responsável por: implementar a interface AIDL, consumir da HAL do acelerômetro e registrar o serviço no Binder.

Etapa 3 — Configuração do Build

1. Adicionar no Android.bp do projeto:

```shell
aidl_interface {
    name: "accel_aidl_<nome_do_grupo>",
    srcs: ["aidl/com/android/accel_<nome_do_grupo>/IAccel.aidl"],
    local_include_dir: "aidl",
    owner: "<nome_do_grupo>",
    unstable: true,
    backend: {
        cpp: {
            enabled: true,
        },
        ndk: {
            enabled: true,
        },
        java: {
            enabled: true,
        },
    },
}
cc_binary {
    name: "accel_daemon_<nome_do_grupo>",
    srcs: [
        "src/accel_daemon_<nome_do_grupo>.cpp",
    ],
    shared_libs: [
        "libacelerometro_hal_<nome_do_grupo>",
        "libbase",
        "liblog",
        "libbinder_ndk",
        "accel_aidl_<nome_do_grupo>-ndk",
    ],
    init_rc: ["accel_<nome_do_grupo>.rc"],
    cflags: [
        "-Wall",
        "-Werror",
    ],
}
```

Etapa 4 — Script de Inicialização

1. Criar arquivo:

```shell
touch accel_<nome_do_grupo>.rc
```

2. Adicionar ao arquivo:

```shell
service accel_daemon_<nome_do_grupo> /system/bin/accel_daemon_<nome_do_grupo>
    class hal
    user root
    group root system
    seclabel u:r:su:s0

    restart_period 5
```

Esse arquivo serve para que o serviço inicie automaticamente no boot.

Etapa 5 — Compilação

```shell
cd /data/aosp/android_course_environment
```

1. Carrega variáveis e funções necessárias para compilação:

```shell
source build/new_envsetup.sh
```

2. Selecione o alvo de compilação:

```shell
lunch aosp_rpi5-bp4a-userdebug
```

3. Compile o módulo compile usando o comando m seguido do nome definido no `cc_binary`:

```shell
croot
```

4. Comando para compilação:

```shell
m accel_daemon_<nome_do_grupo>
```

Baixar arquivos compilados

1. Salve os arquivos gerados para sua máquina local:
    - Biblioteca da HAL:

```shell
/data/aosp/android_course_environment/out_$USER/target/product/rpi5/vendor/lib64/libacelerometro_hal_<nome_do_grupo>.so
```

    - Biblioteca NDK:

```shell
/data/aosp/android_course_environment/out_$USER/target/product/rpi5/system/lib64/accel_aidl_<nome_do_grupo>-ndk.so
```

    - Init:

```shell
/data/aosp/android_course_environment/out_$USER/target/product/rpi5/system/etc/init/accel_<nome_do_grupo>.rc
```

    - Binário do daemon:

```shell
/data/aosp/android_course_environment/out_$USER/target/product/rpi5/system/bin/accel_daemon_<nome_do_grupo>
```

2. Mande os arquivos para a plaquinha via ADB push:

```shell
adb push accel_daemon_<nome_grupo> /system/bin/
adb push accel_aidl_<nome_grupo>-ndk.so /system/lib64/
adb push accel_<nome_do_grupo>.rc /system/etc/init/
adb push libacelerometro_hal_<nome_grupo>.so /system/lib64/
```

```
Documentos/android embarcado/disciplina_android_embarcado_framework
❯ adb push accel_daemon_froyo /system/bin/
accel_daemon_froyo: 1 file pushed, 0 s...ed. 212.5 MB/s (52000 bytes in 0.000s)

Documentos/android embarcado/disciplina_android_embarcado_framework
❯ adb push accel_aidl_froyo-ndk.so /system/lib64/
accel_aidl_froyo-ndk.so: 1 file pushed, 0 skipped. 202.1 MB/s (52112 bytes in 0.000s)

Documentos/android embarcado/disciplina_android_embarcado_framework
❯ adb push accel_froyo.rc /system/etc/init/
accel_froyo.rc: 1 file pushed, 0 skipped. 1.3 MB/s (153 bytes in 0.000s)

Documentos/android embarcado/disciplina_android_embarcado_framework
❯ adb push libacelerometro_hal_froyo.so /system/lib64/
libacelerometro_hal_froyo.so: 1 file pushed, 0 skipped. 42.2 MB/s (35688 bytes in 0.001s)

Documentos/android embarcado/disciplina_android_embarcado_framework
❯
```

3. Reinicie o sistema

```shell
adb reboot
```

4. Conecte-se novamente a plaquinha:

```shell
adb pair <IP_DA_PLAQUINHA>
adb root
adb remount
```

Etapa 6 — Testes

1. Entre no shell:

```shell
adb shell
```

2. Liste os serviços:

```shell
service list
```

Encontre o nome do serviço nessa lista:

```
❯ adb shell
rpi5:/ # service list
Found 283 services:
9       DockObserver: []
1       SurfaceFlinger: [android.ui.ISurfaceComposer]
2       SurfaceFlingerAIDL: [android.gui.ISurfaceComposer]
3       accessibility: [android.view.accessibility.IAccessibilityManager]
4       account: [android.accounts.IAccountManager]
5       acell_froyo: [com.android.accel_froyo.IAccel]
6       activity: [android.app.IActivityManager]
7       activity_task: [android.app.IActivityTaskManager]
8       adb: [android.debug.IAdbManager]
9       adservices_manager: [android.app.adservices.IAdServicesManager]
10      advanced_protection: [android.security.advancedprotection.IAdvancedProtectionService]
11      alarm: [android.app.IAlarmManager]
```

3. Ver logs:

```shell
adb logcat -b all | grep -i accel_daemon_<nome_do_grupo>
```

```
❯ adb logcat -b all | grep -i accel_daemon_froyo
04-02 20:17:15.113   582   582 I accel_daemon_froyo: Iniciando o daemon do acelerômetro...
04-02 20:17:15.114   582   582 I accel_daemon_froyo: HAL do Acelerômetro' inicializada com sucesso.
04-02 20:17:15.115   582   582 I accel_daemon_froyo: Serviço registrado como 'acell_native'
```

4. Testar chamadas Binder:

```shell
service call <nome_servico> 1
service call <nome_servico> 2
service call <nome_servico> 3
```

```
rpi5:/ # service call acell_froyo getAccelX
Result: Parcel(Error: 0xfffffffffffffffb6 "Not a data message")
rpi5:/ # service call acell_froyo 1
Result: Parcel( 00000000 bc6edcc3   '......n.')
rpi5:/ # service call acell_froyo 2
Result: Parcel( 00000000 3f5fe2f8   '....._?')
rpi5:/ # service call acell_froyo getAccelY
Result: Parcel(Error: 0xfffffffffffffffb6 "Not a data message")
rpi5:/ # service call acell_froyo 2
Result: Parcel( 00000000 3f62d289   '.....b?')
rpi5:/ # service call acell_froyo 3
Result: Parcel( 00000000 beee1cde   '........')
rpi5:/ #
```

```
❯ adb logcat -b all | grep -i accel_daemon_froyo
04-02 20:17:15.113   582   582 I accel_daemon_froyo: Iniciando o daemon do acelerômetro...
04-02 20:17:15.114   582   582 I accel_daemon_froyo: HAL do Acelerômetro' inicializada com sucesso.
04-02 20:17:15.115   582   582 I accel_daemon_froyo: Serviço registrado como 'acell_native'
04-02 20:27:57.121   582   582 I accel_daemon_froyo: Accel X: -0.014579
04-02 20:30:41.538   582   582 I accel_daemon_froyo: Accel Y: 0.874557
04-02 20:31:19.264   582   582 I accel_daemon_froyo: Accel Y: 0.886025
04-02 20:33:28.425   582   582 I accel_daemon_froyo: Accel Z: -0.465064
```

Cada número corresponde a:

- 1 ⟶ getAccelX
- 2 ⟶ getAccelY
- 3 ⟶ getAccelZ

Etapa 7 — Relatório de Entrega

Seu relatório final deve conter a explicação dos seguintes itens:

- ☐ **Visão Geral da Arquitetura**

  A arquitetura do Android separa as aplicações do hardware subjacente através de camadas bem definidas. O fluxo de comunicação implementado nesta prática segue o modelo:

  - ☐ HAL → Serviço (Binder) → Aplicativo

  HAL (Hardware Abstraction Layer): Camada mais baixa no espaço de usuário, responsável por interagir diretamente com o driver I2C do acelerômetro e ler os dados brutos.

  Serviço (Binder): Atua como um processo em segundo plano (Daemon) que inicializa e consome a HAL. Ele expõe as capacidades da HAL para o resto do sistema operacional utilizando o Binder, que é o mecanismo principal de Comunicação Interprocessos (IPC) do Android.

  Aplicativo (Cliente): Qualquer aplicação ou ferramenta de terminal (neste caso, o comando `service call`) que atua como cliente. O cliente solicita os dados de aceleração fazendo uma chamada remota (RPC) ao serviço através do driver do Binder no kernel, sem precisar conhecer os detalhes físicos do sensor.

- ☐ **Interface AIDL** (IAccel.aidl)

  A Android Interface Definition Language (AIDL) foi utilizada para definir o contrato de comunicação entre o cliente e o serviço. O arquivo define a interface `IAccel` com os métodos `getAccelX()`, `getAccelY()` e `getAccelZ()`, garantindo que o sistema de build gere automaticamente os *stubs* e *proxies* necessários para a conversão (marshalling/unmarshalling) dos dados via Binder.

- ☐ **Serviço** (accel_daemon_<nome_grupo>.cpp)

  Escrito em C++, este arquivo contém a implementação real dos métodos definidos no AIDL. Ele herda de `BnAccel`, instancia a classe `AcelerometroHAL` e registra o serviço no `ServiceManager` do Android utilizando a biblioteca NDK do Binder. Ele também gerencia o *thread pool* necessário para receber requisições assíncronas concorrentes.

- ☐ **Build** (Android.bp)

  O script de compilação do sistema Soong. Ele possui duas funções nesta prática: usar o módulo `aidl_interface` para compilar o arquivo AIDL nos backends suportados (C++, NDK, Java) e

usar o módulo `cc_binary` para gerar o executável do daemon, linkando as bibliotecas compartilhadas da HAL (`libacelerometro_hal`) e do Binder (`libbinder_ndk`).

- ☐ **Init** (accel_<nome_do_grupo>.rc)

  O processo `init` do Android lê arquivos de configuração `.rc` durante a inicialização do sistema. Este script configura o daemon do acelerômetro para ser iniciado automaticamente no boot como um serviço da classe `hal`, definindo seus privilégios de usuário (root) e seu contexto de segurança SELinux (`u:r:su:s0`).

- ☐ **Dificuldades Encontradas:** Relato sobre eventuais problemas com:
  - ☐ I2C
  - ☐ Permissões
  - ☐ Binder
  - ☐ Build